

Proyecto Intermodular: Sistema de Copias de Seguridad y Recuperación de Datos para una Empresa

1. Introducción

En los últimos años han aumentado los ciberataques, fallos de hardware y pérdidas de datos en empresas grandes y pequeñas. La mayoría de estos problemas provoca interrupciones, pérdidas económicas y, en ocasiones, la imposibilidad de recuperar información importante.

Para solucionarlo, planteamos un sistema completo de copias de seguridad (backup) y de recuperación de datos que una empresa pueda utilizar para protegerse. Este proyecto combina conocimientos de Redes Locales, Sistemas Operativos Monopuesto, Mantenimiento de Sistemas Microinformáticos y Aplicaciones Web, mostrando cómo se integran varias asignaturas para resolver un problema real.

2. Objetivos del proyecto

Diseñar una red local donde un servidor almacene las copias de seguridad de los equipos.

Configurar un sistema de backup automático usando Cobian Reflector.

Crear un servidor accesible desde la LAN para guardar las copias.

Automatizar la supervisión del backup mediante un script en Python y avisos por correo.

Documentar el proceso de instalación, configuración y pruebas.

Desarrollar una pequeña página web informativa para mostrar el estado del servidor o el funcionamiento del sistema (opcional).

Aplicar protocolos de mantenimiento preventivo y correctivo.

3. Integración de módulos 3.1 Redes Locales

Diseño de una pequeña red LAN con:

Servidor de backups.

Equipos cliente que enviarán sus datos.

Carpeta compartida en el servidor mediante SMB.

Configuración de:

IP estática en el servidor.

Permisos de acceso.

Firewall y políticas de red básicas.

Estructura de red propuesta:

Servidor Backup → IP: 192.168.1.50

Cientes → DHCP o IP fija

Carpeta compartida: \192.168.1.50\Backups

3.2 Sistemas Operativos Monopuesto

Instalación y configuración de:

Cobian Reflector en Windows.

Carpetas origen del backup en cada equipo.

Cifrado de los archivos (AES-256).

Programación de tareas:

Frecuencia diaria o semanal.

Compresión ZIP.

Comprobación de permisos locales y correctos accesos a la unidad de red.

3.3 Mantenimiento de Sistemas Microinformáticos

En este módulo se aplica:

Creación de un plan de mantenimiento preventivo:

Verificar espacio en disco del servidor.

Controlar la integridad del backup.

Comprobar que los servicios de red sigan activos.

Simulación de fallos:

Borrado accidental de archivos para demostrar la restauración.

Uso de logs para detectar errores.

Automatización con un script de Python que envía alertas si algo falla.

3.4 Aplicaciones Web

Opcional pero ideal para integrarlo:

Desarrollo de una página web sencilla (HTML/PHP) que muestre:

Último backup realizado.

Estado del servidor.

Acceso rápido a logs.

La web se alojaría en el mismo servidor o en otro dispositivo de la red.

4. Tecnologías utilizadas

Cobian Reflector → Copias automáticas.

Windows / Linux → Según el sistema del servidor.

Python + smtplib → Envío de alertas por email.

ZIP + AES-256 → Compresión y cifrado seguro.

SMB / Carpetas compartidas → Almacenamiento en red.

HTML / CSS / PHP (opcional) → Mini panel web de supervisión.

5. Configuración del sistema de backup

Origen

C:\DatosEmpresa

(Carpetas con información importante)

Destino

Servidor:

\ServidorBackup\CopiaSeguridad

Parámetros

Compresión: ZIP

Cifrado: AES-256

Frecuencia: diaria

Registros: C:\Logs\CobianBackup

6. Script de supervisión en Python

Revisa el log del respaldo y, si detecta un fallo, envía un correo al administrador.

```
import smtplib from email.mime.text import MIMEText
```

```
LOG_PATH = r"C:\Logs\Cobian\backup_log.txt" EMAIL_FROM = "alertas@tuservidor.com" EMAIL_TO =  
"admin@empresa.com" SMTP_SERVER = "smtp.tuservidor.com" SMTP_PORT = 587 SMTP_USER =  
"alertas@tuservidor.com" SMTP_PASS = "contraseña_segura"
```

```
def leer_log(): with open(LOG_PATH, "r", encoding="utf-8") as f: return f.read()
```

```
def detectar_error(contenido): palabras_clave = ["Error", "Failed", "Warning", "No se pudo"] return any(palabra  
in contenido for palabra in palabras_clave)
```

```
def enviar_alerta(mensaje): msg = MIMEText(mensaje) msg["Subject"] = "Falla detectada en el backup"  
msg["From"] = EMAIL_FROM msg["To"] = EMAIL_TO
```

```
with smtplib.SMTP(SMTP_SERVER, SMTP_PORT) as server:  
    server.starttls()
```

```
server.login(SMTP_USER, SMTP_PASS)
server.sendmail(EMAIL_FROM, EMAIL_TO, msg.as_string())
```

7. Pruebas y resultados

Se realizan pruebas para asegurar que el sistema funciona:

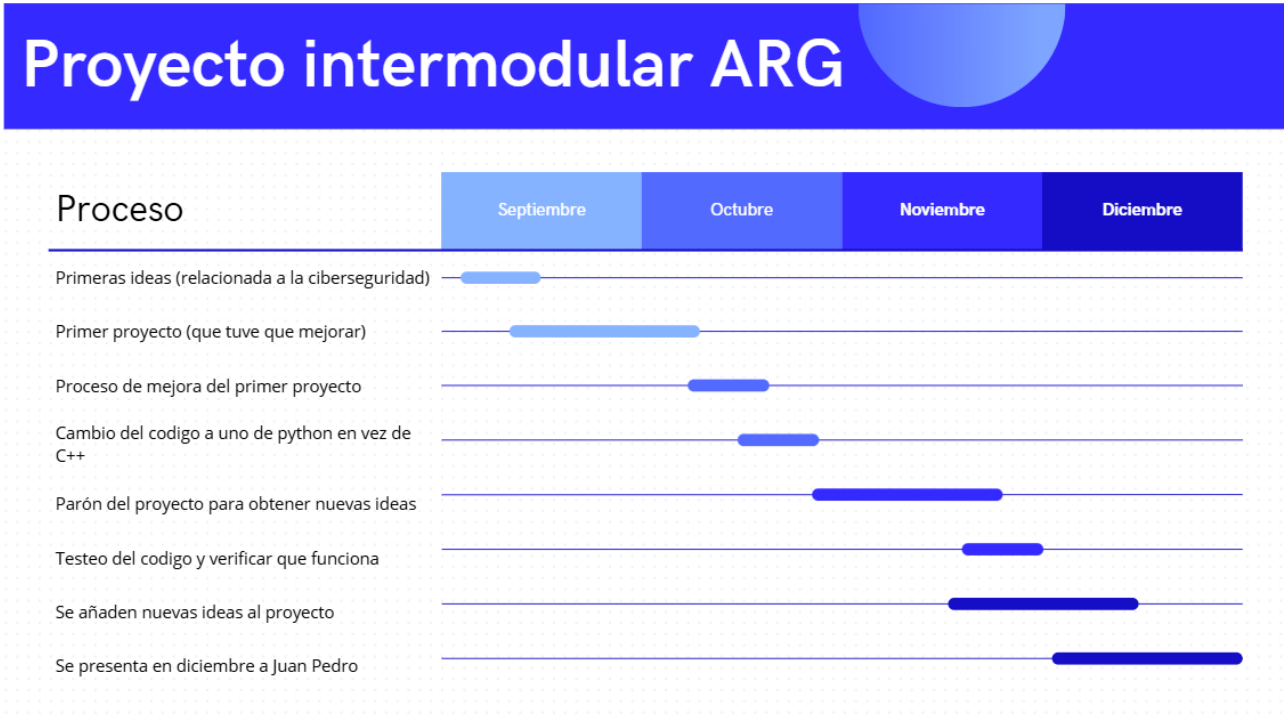
Prueba 1: El equipo genera una copia y llega correctamente al servidor.

Prueba 2: Se borra un archivo importante simulando un ataque → se restaura desde el servidor sin problemas.

Prueba 3: El script detecta un error y envía un email de alerta.

Los resultados fueron correctos: el sistema responde bien a fallos y permite recuperar datos sin complicaciones.

8. Diagrama de gantt



9. Conclusiones

El sistema creado demuestra que es posible implementar una solución de backup segura, económica y fácil de mantener en una pequeña empresa o centro educativo.

El proyecto combina varias áreas del ciclo:

Redes para compartir recursos y asegurar accesos.

Sistemas operativos para instalar y configurar el software.

Mantenimiento para revisar el estado de las copias.

Aplicaciones web para tener un panel de supervisión.

Con esta integración, se obtiene un sistema completo que protege la información sensible y permite recuperarla rápidamente en caso de fallo.